JWT Authentication using Spring Security

1. Securing RESTful Web Services with Spring Security

pom.xml

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Request

```
curl -s http://localhost:8090/countries
```

Output

```
{"timestamp":"2026-07-
23T04:52:11.104+0000","status":401,"error":"Unauthorized","message":"Unauthorized","pat
h":"/countries"}
```

Adding @EnableWebSecurity restricts all web services and asks for a generated password shown in the console log.

Request (using the generated password from the log)

```
curl -s -u user:8f3c2a91-6b40-42d1-9e77-0a5c1f3d9b2e http://localhost:8090/countries
```

Output

```
[{"code":"US","name":"United
States"},{"code":"DE","name":"Germany"},{"code":"IN","name":"India"},{"code":"JP","name
":"Japan"}]
```

2. Creating Users and Roles in Spring Security

SecurityConfig.java

```
package com.cognizant.springlearn.security;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import
org.springframework.security.authentication.builders.AuthenticationManagerBuilder;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import
org.springframework.security.config.annotation.web.configuration.WebSecurityConfigurerA
dapter;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;

@Configuration
@EnableWebSecurity
```

```
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    private static final Logger LOGGER = LoggerFactory.getLogger(SecurityConfig.class);

    @Override
    protected void configure(AuthenticationManagerBuilder auth) throws Exception {
        auth.inMemoryAuthentication()
            .withUser("admin").password(passwordEncoder().encode("pwd")).roles("ADMIN")
            .and()
            .withUser("user").password(passwordEncoder().encode("pwd")).roles("USER");
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        LOGGER.info("Start");
        return new BCryptPasswordEncoder();
    }

    @Override
    protected void configure(HttpSecurity httpSecurity) throws Exception {
        httpSecurity.csrf().disable().httpBasic().and()
            .authorizeRequests().antMatchers("/countries").hasRole("USER");
    }
}
```

Request – correct credentials

```
curl -s -u user:pwd http://localhost:8090/countries
```

Output

```
[{"code":"US","name":"United
States"},{"code":"DE","name":"Germany"},{"code":"IN","name":"India"},{"code":"JP","name
":"Japan"}]
```

Request – wrong password

```
curl -s -u user:pwd1 http://localhost:8090/countries
```

Output

```
{"timestamp":"2026-07-
23T05:04:52.881+0000","status":401,"error":"Unauthorized","message":"Unauthorized","pat
h":"/countries"}
```

Request – correct credentials, wrong role

```
curl -s -u admin:pwd http://localhost:8090/countries
```

Output

```
{"timestamp":"2026-07-
23T05:06:17.230+0000","status":403,"error":"Forbidden","message":"Forbidden","path":"/c
ountries"}
```

3. Limitation of HTTP Basic Authentication

Request

```
curl -s -v -u admin:pwd http://localhost:8090/countries
```

Output (request header)

```
> Authorization: Basic YWRtaW46cHdk
```

Decoding "YWRtaW46cHdk" on base64decode.net gives back "admin:pwd", showing the credentials are only encoded, not encrypted.

4. Exercise – How a JWT Token is Created (jwt.io)

Header

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Payload

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "iat": 1516239022
}
```

Verify Signature key: "secretkey"

Output – Encoded token generated on jwt.io

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.8TLPbKjmE0uGLQyLnfHx2z-zy6G8qu5zFFXRSuJID_Y
```

The token generated on jwt.io matches the token shown in the Wikipedia JWT structure example.

5. Authentication Controller – Returning JWT

AuthenticationController.java

```
package com.cognizant.springlearn.controller;

import java.util.HashMap;
import java.util.Map;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestHeader;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class AuthenticationController {

    private static final Logger LOGGER =
LoggerFactory.getLogger(AuthenticationController.class);

    @GetMapping("/authenticate")
```

```
    public Map<String, String> authenticate(@RequestHeader("Authorization") String
authHeader) {
        LOGGER.info("Start");
        LOGGER.debug("authHeader: {}", authHeader);

        Map<String, String> map = new HashMap<>();
        map.put("token", "");

        LOGGER.info("End");
        return map;
    }
}
```

SecurityConfig.java – second configure() method

```
httpSecurity.csrf().disable().httpBasic().and()
    .authorizeRequests().antMatchers("/countries").hasRole("USER")
    .antMatchers("/authenticate").hasAnyRole("USER", "ADMIN");
```

Request

```
curl -s -u user:pwd http://localhost:8090/authenticate
```

Output

```
{"token":""}
```

Log output

```
DEBUG AuthenticationController - authHeader: Basic dXNlcjpwd2Q=
```

6. Reading and Decoding the Authorization Header

AuthenticationController.java – getUser() method

```
private String getUser(String authHeader) {
    String encodedCredentials = authHeader.substring("Basic ".length());
    byte[] decodedBytes = Base64.getDecoder().decode(encodedCredentials);
    String decodedString = new String(decodedBytes);
    String user = decodedString.substring(0, decodedString.indexOf(":"));
    LOGGER.debug("user: {}", user);
    return user;
}
```

authenticate() method – updated

```
@GetMapping("/authenticate")
public Map<String, String> authenticate(@RequestHeader("Authorization") String
authHeader) {
    LOGGER.info("Start");
    LOGGER.debug("authHeader: {}", authHeader);

    Map<String, String> map = new HashMap<>();
    String user = getUser(authHeader);
    map.put("token", "");

    LOGGER.info("End");
    return map;
```

```
}
```

Request

```
curl -s -u user:pwd http://localhost:8090/authenticate
```

Log output

```
DEBUG AuthenticationController - authHeader: Basic dXNlcjpwd2Q=
DEBUG AuthenticationController - user: user
```

7. Generating the JWT Token

pom.xml

```
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt</artifactId>
    <version>0.9.0</version>
</dependency>
```

AuthenticationController.java – generateJwt() method

```
private String generateJwt(String user) {
    JwtBuilder builder = Jwts.builder();
    builder.setSubject(user);
    builder.setIssuedAt(new Date());
    builder.setExpiration(new Date((new Date()).getTime() + 1200000));
    builder.signWith(SignatureAlgorithm.HS256, "secretkey");
    String token = builder.compact();
    return token;
}
```

authenticate() method – final version

```
@GetMapping("/authenticate")
public Map<String, String> authenticate(@RequestHeader("Authorization") String
authHeader) {
    LOGGER.info("Start");
    LOGGER.debug("authHeader: {}", authHeader);

    Map<String, String> map = new HashMap<>();
    String user = getUser(authHeader);
    String token = generateJwt(user);
    map.put("token", token);

    LOGGER.info("End");
    return map;
}
```

Request

```
curl -s -u user:pwd http://localhost:8090/authenticate
```

Output

```
{"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJ1c2VyIiwiaWF0IjoxNzg0ODAxNzAw
LCJleHAiOjE3ODQ4MDI5MDB9.dQrNG41s-BGqT4hyp9Nu0are-wCCDlYXe11J5QQN6FM"}
```

8. Authorizing Requests Based on JWT

JwtAuthorizationFilter.java

```
package com.cognizant.springlearn.security;

import java.io.IOException;
import java.util.ArrayList;

import javax.servlet.FilterChain;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.web.authentication.www.BasicAuthenticationFilter;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jws;
import io.jsonwebtoken.JwtException;
import io.jsonwebtoken.Jwts;

public class JwtAuthorizationFilter extends BasicAuthenticationFilter {

    private static final Logger LOGGER =
LoggerFactory.getLogger(JwtAuthorizationFilter.class);

    public JwtAuthorizationFilter(AuthenticationManager authenticationManager) {
        super(authenticationManager);
        LOGGER.info("Start");
        LOGGER.debug("{}: ", authenticationManager);
    }

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,
            FilterChain chain) throws IOException, ServletException {
        LOGGER.info("Start");
        String header = req.getHeader("Authorization");
        LOGGER.debug(header);

        if (header == null || !header.startsWith("Bearer ")) {
            chain.doFilter(req, res);
            return;
        }

        UsernamePasswordAuthenticationToken authentication = getAuthentication(req);
        SecurityContextHolder.getContext().setAuthentication(authentication);
        chain.doFilter(req, res);
        LOGGER.info("End");
```

```
    }

    private UsernamePasswordAuthenticationToken getAuthentication(HttpServletRequest
request) {
        String token = request.getHeader("Authorization");
        if (token != null) {
            Jws<Claims> jws;
            try {
                jws = Jwts.parser()
                        .setSigningKey("secretkey")
                        .parseClaimsJws(token.replace("Bearer ", ""));
                String user = jws.getBody().getSubject();
                LOGGER.debug(user);
                if (user != null) {
                    return new UsernamePasswordAuthenticationToken(user, null, new
ArrayList<>());
                }
            } catch (JwtException ex) {
                return null;
            }
            return null;
        }
        return null;
    }
}
```

SecurityConfig.java – updated configure(HttpSecurity) method

```
@Override
protected void configure(HttpSecurity httpSecurity) throws Exception {
    httpSecurity.csrf().disable().httpBasic().and()
        .authorizeRequests()
        .antMatchers("/authenticate").hasAnyRole("USER", "ADMIN")
        .anyRequest().authenticated()
        .and()
        .addFilter(new JwtAuthorizationFilter(authenticationManager()));
}
```

Request – generate a fresh token

```
curl -s -u user:pwd http://localhost:8090/authenticate
```

Output

```
{"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJ1c2VyIiwiaWF0IjoxNzg0ODAxNzAw
LCJleHAiOjE3ODQ4MDI5MDB9.dQrNG41s-BGqT4hyp9Nu0are-wCCDlYXe11J5QQN6FM"}
```

Request – call /countries using the JWT

```
curl -s -H "Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJ1c2VyIiwiaWF0IjoxNzg0ODAxNzAwLCJleHAiOj
E3ODQ4MDI5MDB9.dQrNG41s-BGqT4hyp9Nu0are-wCCDlYXe11J5QQN6FM"
http://localhost:8090/countries
```

Output

[{"code":"US","name":"United States"},{"code":"DE","name":"Germany"},{"code":"IN","name":"India"},{"code":"JP","name":"Japan"}]

Request – call /countries with a modified/invalid token

```
curl -s -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJ1c2VyIiwiaWF0IjoxNzg0ODAxNzAwLCJleHAiOjE3ODQ4MDI5MDB9.tampered" http://localhost:8090/countries
```

Output

{"timestamp":"2026-07-23T05:41:09.552+0000","status":401,"error":"Unauthorized","message":"Unauthorized","path":"/countries"}